МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра Информационные системы

ОТЧЕТ

по лабораторной работе №1

по дисциплине «Программирование»

Тема: Типы данных и их внутреннее представление в памяти

Студент гр. 6372 _____ Шнайдер В.С.

Санкт-Петербург
2026

1. Цель и содержание работы

Целью работы является изучение особенностей машинного представления целых и вещественных чисел, а также освоение методов работы с отдельными битами. В ходе выполнения были проанализированы коды целых чисел, формат IEEE 754 для типов **float** и **double**, специальные значения, битовые операции и способ переинтерпретации данных через **std::memcpy**, что позволило сравнить модели хранения разных типов данных.

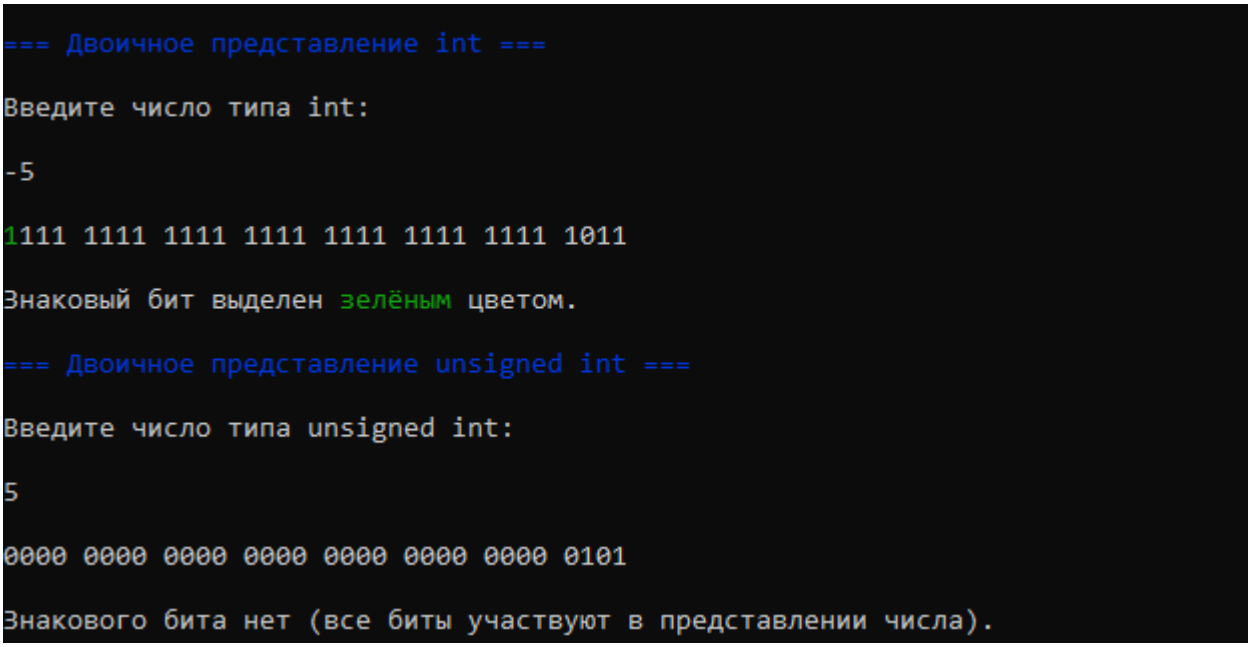
2. Представление целых чисел в памяти

В оперативной памяти целое число занимает фиксированное количество бит, расположенных последовательно. В беззнаковых типах (например, **unsigned int**) каждый бит имеет вес, равный соответствующей степени двойки.
Для знаковых целых чисел необходимо кодировать не только модуль, но и знак. Существует три основных способа кодирования: прямой, обратный и дополнительный. В современных вычислительных системах для знаковых целых типов практически повсеместно применяется дополнительный код, поскольку он устраняет проблему «двойного нуля» и делает аппаратную реализацию арифметических операций более простой.

Положительные числа в дополнительном коде записываются так же, как и в обычном двоичном. Чтобы получить запись отрицательного числа, нужно инвертировать все биты его модуля и прибавить к результату единицу. Например, для 8-разрядного представления числа -5: прямой код - 10000101, обратный - 11111010, дополнительный - 11111011.

В знаковых целых числах самый старший разряд является знаковым: ноль означает положительное число, единица — отрицательное. В разработанной программе биты целого числа выводятся через **std::bitset<32>**, что позволяет безопасно отобразить все 32 позиции от старшего к младшему. При этом знаковый разряд подсвечивается отдельным цветом для наглядности. Также в программе реализован отдельный вывод для типа **unsigned int**, чтобы

продемонстрировать, что у беззнаковых чисел знакового бита нет, и все 32 разряда участвуют в формировании значения.



3. Формат IEEE 754 для вещественных чисел

Числа с плавающей точкой типов **float** и **double** в памяти организованы согласно стандарту IEEE 754. В этом формате двоичное представление делится на три составные части: знаковый бит, поле экспоненты и поле мантиссы.

3.1. Нормализованные и денормализованные значения

Для нормализованного числа экспонента принимает значения, отличные от всех нулей и всех единиц. При этом старшая единица мантиссы подразумевается, но в памяти явно не сохраняется - такой приём называют скрытой (неявной) единицей.

Когда поле экспоненты целиком заполнено нулями, а мантисса при этом не равна нулю, число считается денормализованным. В этом случае скрытая единица уже не подразумевается, что даёт возможность записывать очень малые величины, близкие к нулю. В программе минимальное положительное денормализованное число было получено через стандартную библиотеку:
float denormFloat = std::numeric_limits<float>::denorm_min(); // ~1.4e-45

3.2. Особые значения

Стандарт IEEE 754 резервирует определённые битовые комбинации под специальные случаи. Если поле экспоненты состоит из одних единиц, а мантисса равна нулю, число интерпретируется как бесконечность. Если же при такой экспоненте мантисса отлична от нуля, значением является **NaN** (Not a Number - «не число»). В коде бесконечность задавалась с помощью **std::numeric_limits<float>::infinity()**, что позволило продемонстрировать это особое значение без рискованной операции деления на ноль.

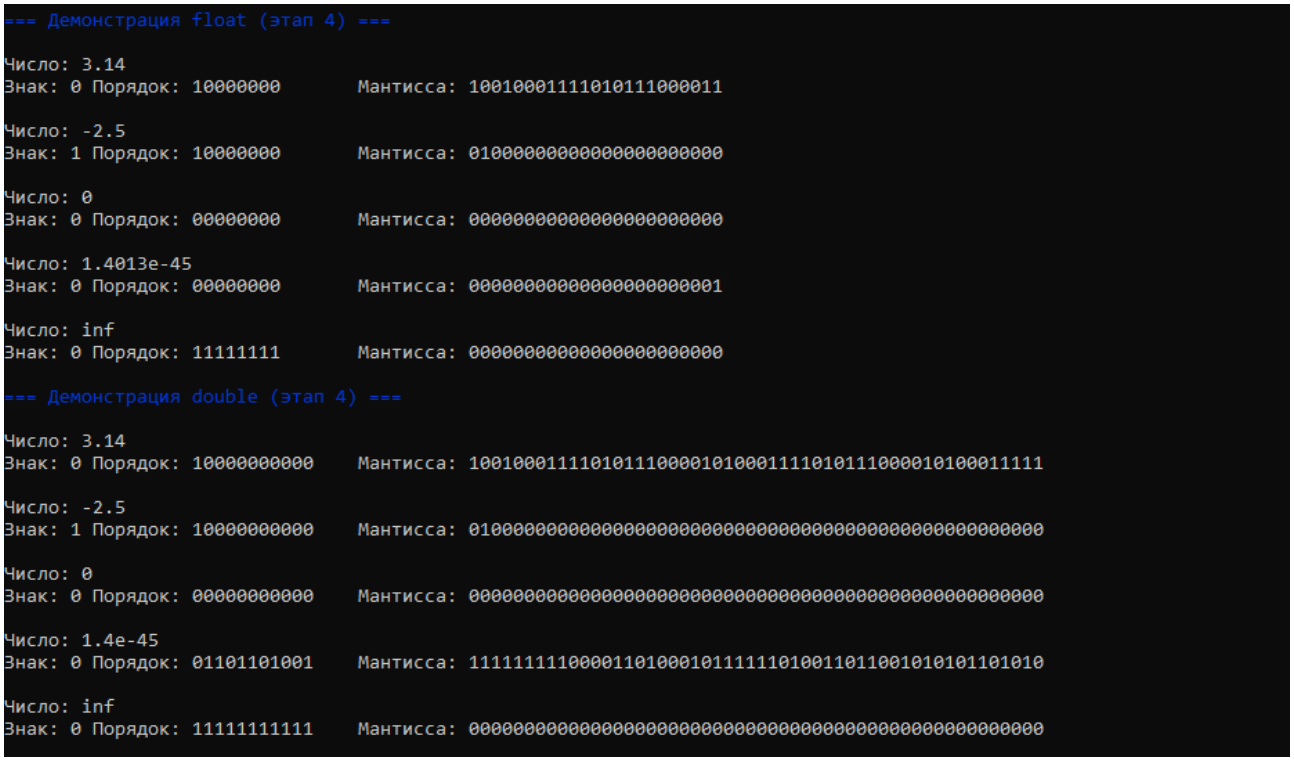
4. Переинтерпретация битов с помощью std::memcpy

Для анализа отдельных полей IEEE 754 необходимо получить не числовое значение после преобразования типа, а точную последовательность битов, уже находящуюся в памяти. В программе для этого используется функция **std::memcpy**.

```
uint32_t bits_f;
std::memcpy(&bits_f, &f, sizeof(f));
```

Функция **std::memcpy** копирует объектное представление байт в байт. Числовое преобразование при этом не выполняется. Для **float** используется **unsigned int**, а для **double** - **unsigned long long**, поскольку в исследуемой системе размеры этих типов совпадают соответственно с 32 и 64 битами. После копирования над целочисленным представлением можно выполнять сдвиги и накладывать маски, что позволяет выделить знак, экспоненту и мантиссу.

В рамках работы была реализована автоматическая демонстрация для нескольких значений: 3.14, -2.5, 0.0, минимального денормализованного числа и бесконечности (**inf**), как для **float**, так и для **double**.



5. Битовые операции, применённые в работе

Для работы с отдельными разрядами использовалась маска **1ULL << k**, где **k** - номер бита. Сдвиг переносит единственную единицу в требуемую позицию. Далее применяются логические побитовые операции:

value |= (1ULL << k) - операция OR принудительно делает выбранный разряд равным 1.

value &= ~(1ULL << k) - инвертированная маска содержит 0 только в выбранной позиции, поэтому AND сбрасывает этот бит.

value ^= (1ULL << k) - XOR с единицей меняет 0 на 1 и наоборот.

Данные операции оформлены в виде отдельной функции **modifyBit**, которая принимает значение, номер бита и тип операции (1 - установить, 0 - сбросить, -1 - инвертировать). Демонстрация проводилась на целых числах, а также на битовом представлении **float** и **double**.

6. Индивидуальное задание: Умножение на 2 через битовый сдвиг

В двоичной системе счисления каждый разряд имеет вес, равный степени двойки. При сдвиге всех битов числа влево на одну позицию вес каждого установленного бита увеличивается в два раза. Поэтому сдвиг влево на 1 эквивалентен умножению числа на 2, если результат помещается в используемый тип данных.

Например: $5 = 00000101_2$. После сдвига влево на один разряд получается 00001010_2, то есть 10.

Результаты на тестовых примерах:

1. Ввод: 5 - Результат: `10`
2. Ввод: 7 - Результат: `14`
3. Ввод: 12 - Результат: `24`
4. Ввод: 100 - Результат: `200`

Во всех тестах после сдвига влево на одну позицию числовое значение увеличилось ровно в два раза.

7. Структура программы и рефакторинг

В соответствии с требованиями к работе, исходный код был разделён на модули:

bit_utils.hpp - заголовочный файл с объявлениями всех функций и директивой **#pragma once**, предотвращающей повторное подключение.

bit_utils.cpp - файл с реализацией функций.

main.cpp - файл, содержащий только функцию **main** с последовательными вызовами всех этапов работы.

Для предотвращения копирования тяжёлых объектов и в целях демонстрации использования константных ссылок, в код была добавлена вспомогательная функция вывода заголовков:

```
void printHeader(const std::string& title);
```

Она принимает строку по константной ссылке и используется для оформления вывода.

8. Выводы

В ходе работы были изучены два принципиально разных способа кодирования числовых данных. Целые знаковые числа представлены в дополнительном коде, который удобен для выполнения арифметики на уровне процессора. Вещественные значения представлены по **IEEE 754** и состоят из знака, экспоненты и мантиссы. Формат дополнительно поддерживает **денормализованные** значения, бесконечности и **NaN**.

Для исследования битового представления **float** и **double** был использован **std::memcpy**. Этот способ не выполняет числового преобразования, а переносит объектное представление байт в байт в целочисленный тип подходящего размера. После этого с помощью сдвигов и масок можно безопасно анализировать или изменять отдельные биты.

Практические эксперименты с установкой, сбросом и инверсией разрядов показали, что одинаковые побитовые операции имеют разный смысл для разных типов данных: у **int** они непосредственно изменяют двоичное целое значение, а у **float** и **double** могут затрагивать знак, экспоненту или мантиссу. В рамках индивидуального задания было реализовано умножение на 2 только через битовый сдвиг **value << 1**, что подтверждает эквивалентность сдвига и умножения на степень двойки.